We have already discussed optimizing a cost function to find the cheapest solution to a well-defined problem, and this is a category of algorithms that, whenever we can describe a derivable cost function, greatly benefit from the application of gradient descent.

Lately, though, a different category of algorithms using gradient descent has gained popularity: learning algorithms.

I've always found the name machine learning a bit deceptive, because it somehow suggests this branch involves machines that can learn in the same way humans do – which is not the case, unfortunately, although there are some similarities.

When we apply gradient descent to solving a problem like traveling salesman or graph embedding, we have a static (usually huge) domain and our goal is to find a point in that domain; in machine learning, instead, we have a dataset, and we want to make sense of it by "learning" a model that describe the dataset and (more importantly) generalize to inputs that were not in the dataset.